

End-to-End Data Analytics Platform

Microsoft Fabric | Public Data | Spain Indicators

A complete data engineering portfolio project covering ingestion, transformation, warehousing, and reporting of Spanish public data sources using the Microsoft Fabric ecosystem.

AUTHOR	STACK	DATE
Ssr Data Analyst	Microsoft Fabric	March 2026
6 years experience	Lakehouse Warehouse	Duration: 1-2 months
Microsoft Stack	PySpark Power BI	Version 1.0

1. Executive Summary

This document defines the scope, objectives, and strategic decisions for a complete end-to-end data analytics project built on **Microsoft Fabric**. The project ingests, transforms, models, and visualizes public indicator data from Spain using five diverse data sources across multiple formats (XLS, XLSX, CSV, REST API, and SQL Server).

The primary goal is twofold: to gain **hands-on practical experience** with Microsoft Fabric beyond certification theory (DP-600), and to build a **professional portfolio** that demonstrates real-world data engineering capabilities for the freelance market in Europe and the United States.

Every component of this project is designed and documented as if it were a client deliverable: clean code, justified technical decisions, professional documentation, and a well-structured GitHub repository.

2. Professional Profile

Role	Ssr Data Analyst Transitioning to Analytics Engineer
Experience	6 years in data analytics across consulting firms (Argentina & Spain)
Location	Barcelona, Spain (Full Remote)
Certifications	PL-300 (Power BI Data Analyst) DP-600 (Fabric Analytics Engineer)
Core Stack	Power BI, SQL Server, SSIS, PowerShell, Azure Data Factory, Microsoft Fabric
Target	Freelance Analytics Engineer (Remote EU/USA)

Relevant Background

For the past two years, the author has led a project for a public entity in Spain, ingesting and normalizing 80+ public data sources (CSV, legacy Excel, XLSX, REST API, web scraping) into a SQL Server data warehouse with a star schema model (fact table, dimension tables for indicators, autonomous communities, and calendar). The solution was orchestrated with SSIS in a fully on-premise environment and delivered through Power BI dashboards and web applications.

This portfolio project takes that accumulated domain knowledge and re-implements a curated subset of the solution using Microsoft Fabric's modern cloud-native stack, demonstrating the ability to migrate and modernize on-premise data platforms.

3. Project Scope

What This Project Covers

- Ingestion of 5 carefully selected public data sources covering different formats and access patterns.
- Bronze-Silver-Gold medallion architecture implemented in Microsoft Fabric Lakehouse.
- Data transformation using PySpark Notebooks and Dataflows Gen2, each applied where architecturally appropriate.
- Star schema dimensional model in Fabric Warehouse for analytical querying.
- Power BI report integrated within the Fabric ecosystem (Direct Lake connectivity).
- End-to-end orchestration using Data Pipelines.
- Complete professional documentation (functional and technical) delivered as portfolio-ready artifacts.

What This Project Does NOT Cover

- Production deployment or real-time data refresh schedules.
- User access management, RLS, or multi-tenant configurations.
- The full 80+ source dataset from the original client project.
- Advanced data science or machine learning workloads.

4. Technology Stack & Key Decisions

Component	Technology	Justification
Storage	OneLake (Lakehouse)	Unified storage layer, Delta Lake format, medallion architecture
Ingestion	Data Pipelines	Orchestration of diverse sources with scheduling capabilities
Transformation	Notebooks (PySpark)	Complex transformations, data cleansing, schema enforcement
Light ETL	Dataflows Gen2	Simple transformations where Spark overhead is unnecessary
Data Warehouse	Fabric Warehouse	Star schema model, SQL-based analytical layer
Reporting	Power BI (Direct Lake)	Native Fabric integration, optimal performance
Version Control	GitHub	Code, documentation, and portfolio presentation

Key Architectural Principle: Each Fabric component is used where it naturally fits. Notebooks handle complex PySpark transformations, Dataflows Gen2 handle simple mappings, and Pipelines orchestrate the flow. Nothing is forced to demonstrate tool usage; instead, each choice is justified by the nature of the data and the transformation required.

5. Licensing Strategy

The development environment leverages a **Microsoft Fabric Trial Capacity** (60 days), which provides access to all Fabric workloads with up to 64 Capacity Units and 1 TB of OneLake storage. This capacity is independent of any organizational resources.

Capacity Management Strategy

- **Pre-development planning (Phases 0-4):** Completed before activating the trial to maximize the 60-day window for hands-on development.
- **Trial activation timing:** Triggered only when development begins (Phase 5), ensuring no trial days are wasted on planning activities.
- **Post-trial preservation:** All code, documentation, screenshots, and architecture diagrams are stored in GitHub, ensuring the portfolio persists independently of the Fabric workspace.
- **Trial extension:** If needed, the trial can be extended for an additional 60 days when fewer than 7 days remain.

6. Project Phases & Deliverables

0	Context & Strategic Decisions Deliverable: Project Brief (PDF)	COMPLETE
1	Requirements & Source Selection Deliverable: Requirements Document + Source Catalog	PENDING
2	Architecture Design Deliverable: Architecture Diagram + Design Document	PENDING
3	Functional Specification Deliverable: Functional Document (PDF)	PENDING
4	Technical Specification Deliverable: Technical Document (PDF)	PENDING
5	Development in Fabric Deliverable: Working Solution + Screenshots + Code	PENDING

6

GitHub Publication

Deliverable: Complete Repository + Professional README

PENDING

Estimated Timeline: 6-8 weeks

Phase	Duration	Fabric Required
Phases 0-4 (Planning & Design)	2-3 weeks	No
Phase 5 (Development)	3-4 weeks	Yes (Trial active)
Phase 6 (Publication)	1 week	No

7. Documentation Strategy

All project documentation follows a dual-format approach optimized for both GitHub browsability and professional client delivery:

Format	Purpose	Location
README.md	Landing page: project overview, architecture, quick navigation	Repository root
PDF Documents	Formal deliverables: functional spec, technical spec, project brief	/docs folder
Inline Comments	Code documentation in notebooks and scripts	Within code files
Screenshots	Visual evidence of Fabric workspace, pipelines, reports	/docs/screenshots
Architecture Diagrams	Visual system design and data flow documentation	/docs/diagrams

8. Quality Standards

This project adheres to professional standards as if it were a real client engagement:

- **Code Quality:** Clean, well-commented code following consistent naming conventions across all Fabric components.
- **Decision Transparency:** Every architectural and technical decision is documented with its rationale, including why alternatives were not chosen.
- **Reproducibility:** While the Fabric workspace is ephemeral (tied to the trial capacity), all code, configurations, and visual evidence are preserved in the repository.
- **Professional Presentation:** Documentation quality matches what would be delivered to a paying client, reinforcing the freelance-ready positioning of this portfolio.

9. Repository Structure (Planned)

```
project-name/  
|-- README.md  
|-- /docs  
|   |-- project-brief.pdf  
|   |-- functional-spec.pdf  
|   |-- technical-spec.pdf  
|   |-- /diagrams  
|   |-- /screenshots  
|-- /src  
|   |-- /notebooks  
|   |-- /pipelines  
|   |-- /dataflows  
|   |-- /warehouse  
|   |-- /power-bi  
|-- /data
```

```
|    |-- /raw-samples  
|-- LICENSE  
|-- .gitignore
```

This document serves as the foundational reference for the project. All subsequent phases will build upon the scope, decisions, and standards defined here. The next step is **Phase 1: Requirements & Source Selection**, where the five data sources will be formally defined and justified.

Document version 1.0 | March 2026 | Phase 0 deliverable